? Supervised Fine-Tuning (SFT) of Large Language Models with LoRA

? Supervised Fine-Tuning (SFT) of Large Language Models with LoRA

A Complete Theory + Implementation Tutorial

What youll learn in this notebook:

1. SFT Theory - How pretraining, fine-tuning, and supervised fine-tuning relate. The next-token prediction objective and cross-entropy loss. Why SFT aligns base models to follow instructions.

2. LoRA Theory - Why full fine-tuning is expensive, what Parameter-Efficient Fine-Tuning (PEFT) is, the LoRA rank decomposition (W = W? + BA), where LoRA is injected in transformers, and how to choose hyperparameters.

3. Hands-On Implementation - Load a base model, prepare an instruction-tuning dataset, configure LoRA, train with HuggingFace TRLs , monitor with TensorBoard, run inference, and compare base vs. fine-tuned outputs.

Prerequisites: Familiarity with transformers, attention mechanisms, gradient descent, and PyTorch.

Hardware: This notebook runs on a free Google Colab T4 GPU (16 GB VRAM). We use standard LoRA (16-bit) - not QLoRA.

Key Libraries: , , , , ,

Part 0.0 - Pretraining vs Post-Training: The Big Picture

Before diving into SFT and LoRA, it helps to understand where they sit in the overall LLM development lifecycle. Modern LLMs are not trained in a single step - they go through distinct pretraining and post-training phases, each with fundamentally different goals.

What is Pretraining?

Pretraining is the foundational stage where a model learns language from scratch. The model is trained on massive unlabeled text corpora (trillions of tokens) using self-supervised next-token prediction. The goal is to learn general language understanding - grammar, facts, reasoning patterns, and world knowledge. This stage is enormously expensive (months of GPU time, millions of dollars) and produces a base model that can complete text but cannot follow instructions or hold conversations.

Modern pretraining pipelines are themselves multi-stage. For example, Llama 3.1 used three stages: core pretraining on 15.6T tokens, continued pretraining for context lengthening (8K ? 128K), and a final annealing stage on high-quality data. Similarly, Apples foundation models and Qwen 2 used 2-3 stage pretraining with progressive context extension and data quality refinement.

What is Post-Training?

Post-training is everything that happens after pretraining to make the base model useful. It typically involves two steps:

1. Supervised Fine-Tuning (SFT) - Train on curated (instruction, response) pairs so the model learns to follow instructions and respond helpfully. This is the focus of this notebook.

2. Alignment (RLHF / DPO) - Further refine the model using human preference data so it produces responses that are helpful, harmless, and honest.

Part 0 - Environment Setup

0.1 Install dependencies

Code

```
!pip install -U transformers trl peft datasets accelerate torch tensorboard -q
```

Output

```
 [?25l   ???????????????????????????????????????? 0.0/10.7 MB ? eta -:--:--
????????????????????????????????????????? 10.4/10.7 MB 312.2 MB/s eta 0:00:01
????????????????????????????????????????? 10.7/10.7 MB 155.1 MB/s eta 0:00:00
 [?25h [?25l   ???????????????????????????????????????? 0.0/528.8 kB ? eta -:--:--
????????????????????????????????????????? 528.8/528.8 kB 47.7 MB/s eta 0:00:00
 [?25h [?25l   ???????????????????????????????????????? 0.0/520.7 kB ? eta -:--:--
????????????????????????????????????????? 520.7/520.7 kB 51.6 MB/s eta 0:00:00
 [?25h [?25l   ???????????????????????????????????????? 0.0/383.7 kB ? eta -:--:--
```

```
???????????????????????????????????????? 383.7/383.7 kB 40.0 MB/s eta 0:00:00
???????????????????????????????????????? 5.5/5.5 MB 132.1 MB/s eta 0:00:00
???????????????????????????????????????? 47.6/47.6 MB 52.7 MB/s eta 0:00:00
 [?25hERROR: pip's dependency resolver does not currently take into account all the packages
that are installed. This behaviour is the source of the following dependency conflicts.
tensorflow 2.19.0 requires tensorboard~=2.19.0, but you have tensorboard 2.20.0 which is
incompatible.
```

0.2 Imports

Code

```
!nvidia-smi
```

Output

```
Sat Mar  7 17:52:11 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   43C    P8             11W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Code

```
import torch
import gc
import os
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, PeftModel, get_peft_model
from trl import SFTConfig, SFTTrainer
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
print(f"GPU: {torch.cuda.get_device_name(0)}")
print(f"VRAM: {torch.cuda.get_device_properties(0).total_memory / 1024**3:.1f} GB")
```

Output

```
PyTorch version: 2.10.0+cu128
CUDA available: True
GPU: Tesla T4
VRAM: 14.6 GB
```

0.3 Load TensorBoard extension

We load the TensorBoard Colab extension early so the widget is ready before training begins.

Code

```
%load_ext tensorboard
```

0.4 Utility functions

Code

```
def clear_memory():
"""Free GPU memory between experiments."""
gc.collect()
torch.cuda.empty_cache()
if torch.cuda.is_available():
print(f"GPU allocated: {torch.cuda.memory_allocated() / 1024**3:.2f} GB")
print(f"GPU reserved:  {torch.cuda.memory_reserved() / 1024**3:.2f} GB")
def print_trainable_params(model):
"""Display trainable vs. total parameter counts."""
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total = sum(p.numel() for p in model.parameters())
print(f"Trainable parameters: {trainable:>12,}  ({100 * trainable / total:.4f}%)")
print(f"Total parameters:     {total:>12,}")
def generate_response(model, tokenizer, prompt, max_new_tokens=256):
"""Generate a response from a model given a prompt string."""
messages = [{"role": "user", "content": prompt}]
input_text = tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
inputs = tokenizer(input_text, return_tensors="pt").to(model.device)
with torch.no_grad():
outputs = model.generate(
**inputs,
max_new_tokens=max_new_tokens,
temperature=0.7,
top_p=0.9,
do_sample=True,
pad_token_id=tokenizer.pad_token_id,
)
# Decode only the newly generated tokens
response = tokenizer.decode(outputs[0][inputs["input_ids"].shape[1]:],
skip_special_tokens=True)
return response
```

Part 1 - Supervised Fine-Tuning: Theory

1.1 The LLM training pipeline: from raw text to helpful assistant

Modern large language models go through a multi-stage training pipeline to become the helpful assistants we interact with. Understanding each stage is crucial before we fine-tune one ourselves.

Stage 1: Pretraining

Pretraining is the most computationally expensive stage. The model trains on a massive corpus of unlabeled text (hundreds of billions to trillions of tokens) using a self-supervised next-token prediction objective. This stage costs hundreds of thousands of dollars - roughly $500K+ for a GPT-3-quality model. The result is a base model that has learned language structure, factual knowledge, and reasoning patterns, but has no concept of instruction-following or conversational behavior. It simply predicts the most likely next token.

Stage 2: Supervised Fine-Tuning (SFT)

SFT is the first alignment step. We curate a dataset of high-quality (instruction, response) examples that demonstrate correct model behavior, then fine-tune the base model on these examples using the same next-token prediction loss. The key insight, demonstrated by the LIMA paper (Zhou et al., 2023), is that almost all knowledge is learned during pretraining - SFT merely teaches the model the correct style and format for surfacing that knowledge. This is why SFT is roughly 100× cheaper than pretraining and can achieve remarkable results with as few as 1,000 carefully curated examples.

Stage 3: RLHF / Preference Optimization

After SFT, models are further aligned using Reinforcement Learning from Human Feedback (RLHF) or Direct Preference Optimization (DPO). LLaMA-2 showed that even after high-quality SFT (using 27,540 examples), RLHF yielded massive further improvements in helpfulness and safety.

SFT vs. traditional fine-tuning

It is important to distinguish SFT from classical fine-tuning. Traditional fine-tuning (e.g., fine-tuning BERT for sentiment classification) creates a narrow expert for one specific task. The model may lose its general capabilities. SFT, by contrast, teaches the model a behavioral style - how to follow instructions, respond helpfully, and converse naturally - while preserving its broad knowledge base. The model remains a generalist; it just learns to format its knowledge as a helpful assistant.

1.2 Next-token prediction: the engine behind LLM training

Both pretraining and SFT use the same training objective: next-token prediction via cross-entropy loss. Understanding this objective is essential for understanding what SFT actually does.

How it works

A decoder-only transformer processes a sequence of tokens and, at each position, produces a probability distribution over the entire vocabulary for the next token:

1. Tokenize raw text into discrete tokens using a tokenizer (e.g., BPE)
2. Look up token embeddings + positional embeddings
3. Pass through L stacked transformer decoder blocks (causal self-attention + feed-forward)
4. Apply a linear classification head () ? logits of shape
5. Apply softmax ? probability distribution over vocabulary

Thanks to causal self-attention, each position only attends to itself and preceding tokens. This means the model predicts the next token at every position in a single forward pass.

The mathematical objective

Given a sequence of tokens $\mathbf{x} = [x_1, x_2, \ldots, x_T]$, the language modeling loss (negative log-likelihood / cross-entropy) is:

Where $P_\theta(x_{t+1} \mid x_{\leq t})$ is the probability the model assigns to the correct next token x_{t+1} given all preceding tokens.

Per-token cross-entropy: For a single position, if the model assigns probability p_y to the ground-truth token y:

* If $p_y = 1.0$? loss = 0 (perfect prediction)
* If $p_y = 0.001$? loss = 6.9 (heavily penalized)

The softmax transformation converts raw logits to probabilities:

Token shifting: how labels are constructed

In practice, labels are the same sequence shifted by one position:

At position t, the model sees tokens $[x_1, \ldots, x_t]$ and must predict x_{t+1}. The cross-entropy loss is computed across all positions simultaneously in one forward pass, then averaged.

SFT uses the same loss - with one crucial difference

During SFT, the input sequence typically contains both an instruction (prompt) and a response. The key difference is loss masking: we compute the cross-entropy loss only on the response tokens, not the

instruction tokens. The prompt tokens still pass through the forward pass (providing context), but they dont contribute to the gradient. This teaches the model how to respond to instructions rather than how to reproduce instructions.

1.3 SFT dataset formats: Alpaca, ShareGPT, and ChatML

The structure of your SFT dataset directly determines what the model learns. There are several standard formats:

Alpaca format (single-turn)

The original Alpaca dataset (52K examples generated by GPT-4) uses three fields:

A prompt template wraps these into a single training sequence: Below is an instruction that describes a task. Write a response ### Instruction: {instruction} ### Response: {output}

Conversational / ChatML format (multi-turn)

Most modern models use a conversational format with role-tagged messages:

Each model family has its own chat template with special tokens that delineate speakers and message boundaries. During SFT, you apply (or choose) a chat template for the base model. The TRL handles this automatically.

Prompt-completion format

A simpler format with two fields - and :

The loss is applied only to the completion tokens.

1.4 The cost problem: why full fine-tuning is prohibitive

Full fine-tuning updates every parameter of the model. For large models, this is extremely expensive in both compute and memory.

Memory breakdown for full fine-tuning

For a model with N parameters using AdamW optimizer in mixed precision (fp16 model + fp32 optimizer):

This ~70 GB minimum for a 7B model exceeds the VRAM of any single consumer GPU. A practical rule of thumb: full fine-tuning requires ~10-16 GB of VRAM per billion parameters.

For GPT-3 (175B parameters), each checkpoint alone is ~350 GB. Deploying separate fine-tuned instances for different tasks is prohibitively expensive.

This is the motivation for Parameter-Efficient Fine-Tuning (PEFT).

Part 2 - LoRA: Low-Rank Adaptation Theory

2.1 Parameter-Efficient Fine-Tuning (PEFT) - the big idea

Instead of updating all model parameters, PEFT methods freeze the pretrained weights and train only a small number of additional parameters. This:

* Drastically reduces memory (optimizer states only for tiny parameter set)
* Simplifies deployment (store one base model + small adapter files per task)
* Enables fine-tuning on consumer GPUs (a 7B model with LoRA can be fine-tuned on a single 16GB GPU)

Several PEFT approaches exist:

Adapter Layers (Houlsby et al., 2019) insert small bottleneck feed-forward modules into each transformer block. Limitation: sequential processing adds inference latency.

Prefix Tuning (Li & Liang, 2021) prepends trainable virtual tokens to the input. Limitation: reduces usable context window and is difficult to optimize.

LoRA (Hu et al., 2021) adds trainable low-rank decomposition matrices parallel to existing weights. It is the dominant PEFT method today because it adds zero inference latency (adapters can be merged into the base weights) and is simple, stable, and effective.

2.2 LoRA - the core idea: W' = W? + BA

The weight update formulation

When we fine-tune a pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, the updated weight is:

In full fine-tuning, ΔW is an unconstrained $d \times k$ matrix - the same size as W_0. LoRAs key insight is to constrain ΔW to be low-rank by decomposing it into two smaller matrices:

Where:
- $W_0 \in \mathbb{R}^{d \times k}$ - frozen pretrained weights
- $B \in \mathbb{R}^{d \times r}$ - trainable up-projection matrix
- $A \in \mathbb{R}^{r \times k}$ - trainable down-projection matrix
- $r \ll \min(d, k)$ - the rank (typically 4-64)
- α - scaling hyperparameter
- $\frac{\alpha}{r}$ - effective scaling factor

Forward pass with LoRA

During the forward pass, the output combines the frozen pretrained computation with the LoRA adapter:

The input x is passed through both W_0 (frozen) and the low-rank branch BA (trainable). Their outputs are summed.

Parameter savings

The trainable parameter count drops from $d \times k$ (full) to $r \times (d + k)$ (LoRA):

For GPT-3 (175B parameters), LoRA reduced the trainable set to ~4M parameters - a 10,000× reduction - while matching full fine-tuning performance and reducing GPU memory by 3×.

Initialization

* Matrix A: Initialized from $\mathcal{N}(0, 1/\sqrt{r})$ - small random values

* Matrix B: Initialized to zeros

* This ensures $\Delta W = BA = 0$ at the start of training, so the model begins with its exact pretrained behavior

2.3 Why low-rank works: the intrinsic dimension hypothesis

It may seem surprising that such a drastic rank reduction works. The theoretical justification comes from the intrinsic dimensionality hypothesis (Aghajanyan et al., 2021):

> *Large pretrained models have a low intrinsic dimension - not all parameters are necessary. The weight matrices contain massive redundancy.*

The LoRA authors extended this: the change in weights during model adaptation also has a low intrinsic rank. Empirically, even $r = 1$ or $r = 2$ can produce competitive results when the full rank d is 12,288 (GPT-3).

A 2025 study by Schulman et al. (Thinking Machines Lab) confirmed: LoRA matches full fine-tuning on small-to-medium post-training datasets (instruction tuning, reasoning) when given sufficient rank. The key finding is that LoRAs effectiveness is bounded by its parameter capacity - the number of LoRA parameters should be at least as large as the number of unique completion tokens in the dataset.

2.4 Where LoRA is applied in transformers

Each transformer block contains two main sub-layers with large weight matrices:

1. Multi-Head Attention: query (W_q), key (W_k), value (W_v), and output (W_o) projection matrices

2. Feed-Forward Network (MLP): gate (W_{gate}), up (W_{up}), and down (W_{down}) projection matrices

The original LoRA paper (2021) applied adapters only to the attention query (W_q) and value (W_v)

matrices. However, subsequent research by Raschka et al. and the 2025 LoRA Without Regret study found that applying LoRA to all weight matrices (attention + MLP) yields significantly better results. In fact, the MLP layers matter more than attention: attention-only LoRA with rank 256 underperforms MLP-only LoRA with rank 128 on Llama-3.1-8B.

Modern best practice: apply LoRA to all linear layers.

2.5 LoRA hyperparameters

Rank (r)

Controls the inner dimension of matrices A and B. Lower rank = fewer parameters, faster training, but less expressive updates.
- Typical range: 4-64
- Most common default: 16
- Guideline: Start with $r = 16$. Increase if training loss plateaus; decrease for smaller datasets or faster iteration.

Alpha (α) - scaling factor

The LoRA output is scaled by $\frac{\alpha}{r}$ before being added to the frozen weight output.
- Rule of thumb: Set α to 1× to 2× the rank (e.g., $r = 16, \alpha = 32$)
- The α / r scaling makes the optimal learning rate approximately independent of rank, so you can change rank without retuning LR

Dropout (`lora_dropout`)

Regularization applied within the LoRA branch during training.
- Default: 0.05 (5%)
- Use 0 for speed; increase to 0.1 if overfitting

Target modules

Which weight matrices receive LoRA adapters:
- Minimal (original paper):
- Attention only:
- All linear (recommended):

Learning rate

A critical finding from multiple sources: the optimal learning rate for LoRA is ~10× higher than for full fine-tuning. For SFT: use 2e-4 with LoRA vs. 2e-5 for full fine-tuning.

2.6 Key benefits of LoRA

1. Zero inference latency: After training, merge BA back into W_0: $W = W_0 + \frac{\alpha}{r} BA$. The resulting model has the exact same architecture and inference speed as the original.

2. Cheap task switching: Keep one base model in memory. Swap LoRA adapters (a few MB each) for different tasks by subtracting one BA and adding another - no full model reload needed.

3. Massive memory reduction: Optimizer states, gradients, and checkpoints are only maintained for the tiny LoRA matrices.

4. ~25% faster training throughput: LoRA computes ~2/3 of the FLOPs per forward-backward pass compared to full fine-tuning.

5. Composable: Can combine with other techniques (quantization ? QLoRA, prefix tuning, etc.).

Part 3 - Hands-On Implementation

Now we put theory into practice. We will:
1. Load a base model ()
2. Test it before SFT (it wont follow instructions well)
3. Prepare an instruction-tuning dataset
4. Attach LoRA adapters
5. Train with (logging to TensorBoard)
6. Monitor training live in TensorBoard
7. Compare before vs. after

3.1 Configuration

Code

```
# ============================================================
# Configuration - change these to experiment
# ============================================================
MODEL_NAME = "Qwen/Qwen2.5-1.5B"      # Base model (no -Instruct suffix!)
DATASET_NAME = "yahma/alpaca-cleaned"   # Classic instruction-tuning dataset
NUM_TRAIN_SAMPLES = 2000               # Subset for fast training on Colab
#Maximum number of tokens a Model can process in a single input sequence
MAX_SEQ_LENGTH = 512                   # Maximum sequence length
# LoRA hyperparameters
LORA_R = 16               # Rank
LORA_ALPHA = 32            # Scaling factor (2x rank)
LORA_DROPOUT = 0.05        # Regularization
LORA_TARGET_MODULES = [    # Apply to all linear layers
"q_proj", "k_proj", "v_proj", "o_proj",
"gate_proj", "up_proj", "down_proj",
]
# Training hyperparameters
LEARNING_RATE = 2e-4       # 10x higher than full fine-tuning
NUM_EPOCHS = 1
BATCH_SIZE = 4
GRAD_ACCUM_STEPS = 4       # Effective batch size = 4 * 4 = 16
WARMUP_RATIO = 0.03
OUTPUT_DIR = "./sft-lora-qwen"
LOG_DIR = f"{OUTPUT_DIR}/runs"  # TensorBoard log directory
```

3.2 Load the base model and tokenizer

Code

```
# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
# Ensure pad token is set (base models often lack one)
if tokenizer.pad_token is None:
tokenizer.pad_token = tokenizer.eos_token
tokenizer.pad_token_id = tokenizer.eos_token_id
print(f"Vocabulary size: {len(tokenizer):,}")
print(f"Model max length: {tokenizer.model_max_length:,}")
print(f"Pad token: '{tokenizer.pad_token}' (id={tokenizer.pad_token_id})")
```

Error Output

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart
your session.
You will be able to reuse this secret in all of your notebooks.
```

Output

```
config.json:   0%|          | 0.00/684 [00:00<?, ?B/s]
```

Output

```
tokenizer_config.json: 0.00B [00:00, ?B/s]
```

Output

```
vocab.json: 0.00B [00:00, ?B/s]
```

Output

```
merges.txt: 0.00B [00:00, ?B/s]
```

Output

```
tokenizer.json: 0.00B [00:00, ?B/s]
```

Output

```
Vocabulary size: 151,665
Model max length: 131,072
Pad token: '<|endoftext|>' (id=151643)
```

Code

```
# Load base model in float16 (standard LoRA - not quantized)
model = AutoModelForCausalLM.from_pretrained(
MODEL_NAME,
torch_dtype=torch.float16,
device_map="auto",
trust_remote_code=True,
)
total_params = sum(p.numel() for p in model.parameters())
model_size_gb = sum(p.numel() * p.element_size() for p in model.parameters()) / 1024**3
print(f"\nBase model loaded: {MODEL_NAME}")
print(f"Total parameters: {total_params:,}")
print(f"Model size in memory: {model_size_gb:.2f} GB")
print(f"GPU memory allocated: {torch.cuda.memory_allocated() / 1024**3:.2f} GB")
```

Error Output

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the
```

```
HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
`torch_dtype` is deprecated! Use `dtype` instead!
```

Output

```
model.safetensors:   0%|          | 0.00/3.09G [00:00<?, ?B/s]
```

Output

```
Loading weights:   0%|          | 0/338 [00:00<?, ?it/s]
```

Output

```
generation_config.json:   0%|          | 0.00/138 [00:00<?, ?B/s]
```

Output

```
Base model loaded: Qwen/Qwen2.5-1.5B
Total parameters: 1,543,714,304
Model size in memory: 2.88 GB
GPU memory allocated: 2.88 GB
```

3.3 Test the base model BEFORE fine-tuning

Base models are trained on raw text - they complete text but dont follow instructions. Lets verify.

Code

```
# Test prompts to evaluate instruction-following ability
test_prompts = [
"Explain the concept of recursion in programming in 2-3 sentences.",
"Write a Python function that checks if a number is prime.",
"What are three benefits of regular exercise?",
]
print("=" * 70)
print("BASE MODEL RESPONSES (before SFT)")
print("=" * 70)
for i, prompt in enumerate(test_prompts):
print(f"\n{'?' * 60}")
print(f"Prompt {i+1}: {prompt}")
print(f"{'?' * 60}")
response = generate_response(model, tokenizer, prompt, max_new_tokens=200)
print(f"Response: {response[:500]}")
# Store base responses for later comparison
base_responses = []
for prompt in test_prompts:
base_responses.append(generate_response(model, tokenizer, prompt, max_new_tokens=200))
```

Output

```
======================================================================
BASE MODEL RESPONSES (before SFT)
======================================================================
??????????????????????????????????????????????????????????????
Prompt 1: Explain the concept of recursion in programming in 2-3 sentences.
??????????????????????????????????????????????????????????????
Response: Explain the concept of recursion in programming in 2-3 sentences.ursion in
programming is a technique where a function calls itself to solve a problem. It is a powerful
tool for solving complex problems, but it can also be inefficient and difficult to debug if
not used carefully.
??????????????????????????????????????????????????????????????
Prompt 2: Write a Python function that checks if a number is prime.
??????????????????????????????????????????????????????????????
Response: def is_prime(num):
if num < 2:
return False
for i in range(2, int(num**0.5) + 1):
if num % i == 0:
return False
return True
def is_prime(num):
if num < 2:
return False
for i in range(2, int(num**0.5) + 1):
if num % i == 0:
return False
return True
?
?
?
?
??????????????????????????????????????????????????????????????
Prompt 3: What are three benefits of regular exercise?
??????????????????????????????????????????????????????????????
Response: .CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
```

[Output continues below...]

```
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.Creat
```

3.4 Load and prepare the dataset

Well use the Alpaca dataset - the classic instruction-tuning dataset with ~52K examples. We convert it from Alpaca format to the conversational messages format that expects.

Code

```
# Load dataset
raw_dataset = load_dataset(DATASET_NAME, split="train")
print(f"Full dataset size: {len(raw_dataset):,} examples")
print(f"Columns: {raw_dataset.column_names}")
print(f"\nExample entry:")
print(raw_dataset[0])
```

Output

```
README.md: 0.00B [00:00, ?B/s]
```

Output

```
alpaca_data_cleaned.json:   0%|          | 0.00/44.3M [00:00<?, ?B/s]
```

Output

```
Generating train split:   0%|          | 0/51760 [00:00<?, ? examples/s]
```

Output

```
Full dataset size: 51,760 examples
Columns: ['output', 'input', 'instruction']
Example entry:
{'output': '1. Eat a balanced and nutritious diet: Make sure your meals are inclusive of a
variety of fruits and vegetables, lean protein, whole grains, and healthy fats. This helps to
```

provide your body with the essential nutrients to function at its best and can help prevent chronic diseases.\n\n2. Engage in regular physical activity: Exercise is crucial for maintaining strong bones, muscles, and cardiovascular health. Aim for at least 150 minutes of moderate aerobic exercise or 75 minutes of vigorous exercise each week.\n\n3. Get enough sleep: Getting enough quality sleep is crucial for physical and mental well-being. It helps to

regulate mood, improve cognitive function, and supports healthy growth and immune function. Aim for 7-9 hours of sleep each night.', 'input': '', 'instruction': 'Give three tips for staying healthy.'}

Code

```
def format_alpaca_to_chat(example):
"""
Convert Alpaca format (instruction/input/output) to conversational
messages format for SFTTrainer.
Alpaca format:
{"instruction": "...", "input": "...", "output": "..."}
Target format (ChatML-style messages):
{"messages": [
{"role": "system", "content": "..."},
{"role": "user", "content": "..."},
{"role": "assistant", "content": "..."}
]}
"""
# Combine instruction and input (if present) into user message
if example.get("input") and example["input"].strip():
user_content = f"{example['instruction']}\n\nInput: {example['input']}"
else:
user_content = example["instruction"]
return {
"messages": [
{"role": "system", "content": "You are a helpful, accurate, and concise
assistant."},
{"role": "user", "content": user_content},
{"role": "assistant", "content": example["output"]},
]
}
# Apply formatting and take a subset for training
dataset = raw_dataset.shuffle(seed=42).select(range(NUM_TRAIN_SAMPLES))
dataset = dataset.map(
format_alpaca_to_chat,
remove_columns=raw_dataset.column_names,
desc="Formatting to chat messages",
)
print(f"\nFormatted dataset size: {len(dataset):,}")
print(f"Columns: {dataset.column_names}")
print(f"\nExample formatted entry:")
print(dataset[0])
```

Output

```
Formatting to chat messages:   0%|          | 0/2000 [00:00<?, ? examples/s]
```

Output

```
Formatted dataset size: 2,000
Columns: ['messages']
Example formatted entry:
{'messages': [{'content': 'You are a helpful, accurate, and concise assistant.', 'role':
'system'}, {'content': 'Rearrange the following sentence to make the sentence more
interesting.\n\nInput: She left the party early', 'role': 'user'}, {'content': 'Early, she
```

```
left the party.', 'role': 'assistant'}]}
```

Code

```
# Verify the chat template renders correctly
sample = dataset[0]
rendered = tokenizer.apply_chat_template(sample["messages"], tokenize=False)
print("Rendered chat template (first example):\n")
print(rendered[:800])
```

Output

```
Rendered chat template (first example):
<|im_start|>system
You are a helpful, accurate, and concise assistant.<|im_end|>
<|im_start|>user
Rearrange the following sentence to make the sentence more interesting.
Input: She left the party early<|im_end|>
<|im_start|>assistant
Early, she left the party.<|im_end|>
```

3.5 Configure LoRA adapters

This is where the theory becomes code. We define which layers get LoRA adapters, the rank, scaling factor, and other hyperparameters.

Code

```
# Define LoRA configuration
peft_config = LoraConfig(
r=LORA_R,                          # Rank of decomposition
lora_alpha=LORA_ALPHA,             # Scaling factor (alpha/r applied)
lora_dropout=LORA_DROPOUT,         # Dropout for regularization
bias="none",                       # Don't train bias terms
task_type="CAUSAL_LM",            # Decoder-only language model
target_modules=LORA_TARGET_MODULES, # All attention + MLP layers
)
print("LoRA Configuration:")
print(f"  Rank (r):          {peft_config.r}")
print(f"  Alpha (?):         {peft_config.lora_alpha}")
print(f"  Effective scaling: {peft_config.lora_alpha / peft_config.r}")
print(f"  Dropout:           {peft_config.lora_dropout}")
print(f"  Target modules:    {peft_config.target_modules}")
print(f"  Task type:         {peft_config.task_type}")
```

Output

```
LoRA Configuration:
Rank (r):          16
Alpha (?):         32
Effective scaling: 2.0
Dropout:           0.05
Target modules:    {'up_proj', 'o_proj', 'k_proj', 'v_proj', 'gate_proj', 'down_proj',
'q_proj'}
```

Code

```
# Apply LoRA to the model and inspect
lora_model = get_peft_model(model, peft_config)
print("\n" + "=" * 50)
print("PARAMETER COMPARISON")
print("=" * 50)
print_trainable_params(lora_model)
# Show the LoRA module structure for one layer
print("\n\nLoRA module example (layer 0, q_proj):")
print(lora_model.model.model.layers[0].self_attn.q_proj)
```

Output

```
==================================================
PARAMETER COMPARISON
==================================================
Trainable parameters:   18,464,768  (1.1820%)
Total parameters:     1,562,179,072
LoRA module example (layer 0, q_proj):
lora.Linear(
(base_layer): Linear(in_features=1536, out_features=1536, bias=True)
(lora_dropout): ModuleDict(
(default): Dropout(p=0.05, inplace=False)
)
(lora_A): ModuleDict(
(default): Linear(in_features=1536, out_features=16, bias=False)
)
(lora_B): ModuleDict(
(default): Linear(in_features=16, out_features=1536, bias=False)
)
(lora_embedding_A): ParameterDict()
(lora_embedding_B): ParameterDict()
(lora_magnitude_vector): ModuleDict()
)
```

Compare with Model with no LoRA

Understanding the parameter count

Lets verify the math from Part 2.

Code

```
# Manual calculation of LoRA parameters
# For each target module, LoRA adds: r * (d_in + d_out) parameters
# (A is r x d_in, B is d_out x r)
print("Manual LoRA Parameter Calculation:")
print("-" * 50)
total_lora_params = 0
sample_layer = lora_model.model.model.layers[0]
# Check dimensions of each target module type
for name in LORA_TARGET_MODULES:
# Navigate to the module
if "proj" in name and ("q_" in name or "k_" in name or "v_" in name or "o_" in name):
module = getattr(sample_layer.self_attn, name)
else:
module = getattr(sample_layer.mlp, name)
if hasattr(module, 'lora_A'):
# Get dimensions
A_shape = module.lora_A['default'].weight.shape  # (r, in_features)
B_shape = module.lora_B['default'].weight.shape  # (out_features, r)
params_per_layer = A_shape[0] * A_shape[1] + B_shape[0] * B_shape[1]
print(f"  {name:12s}: A={list(A_shape)}, B={list(B_shape)} ? {params_per_layer:,}
params/layer")
total_lora_params += params_per_layer
num_layers = len(lora_model.model.model.layers)
print(f"\nParameters per transformer block: {total_lora_params:,}")
print(f"Number of transformer blocks:     {num_layers}")
print(f"Total LoRA parameters:            {total_lora_params * num_layers:,}")
```

Output

```
Manual LoRA Parameter Calculation:
--------------------------------------------------
q_proj      : A=[16, 1536], B=[1536, 16] ? 49,152 params/layer
k_proj      : A=[16, 1536], B=[256, 16] ? 28,672 params/layer
v_proj      : A=[16, 1536], B=[256, 16] ? 28,672 params/layer
o_proj      : A=[16, 1536], B=[1536, 16] ? 49,152 params/layer
gate_proj   : A=[16, 1536], B=[8960, 16] ? 167,936 params/layer
up_proj     : A=[16, 1536], B=[8960, 16] ? 167,936 params/layer
down_proj   : A=[16, 8960], B=[1536, 16] ? 167,936 params/layer
Parameters per transformer block: 659,456
Number of transformer blocks:     28
Total LoRA parameters:            18,464,768
```

3.6 Configure the training run

We use TRLs which extends with SFT-specific options. All metrics are logged to TensorBoard for live monitoring.

Code

```
# Remove the PEFT wrapper for now - SFTTrainer will re-apply it via peft_config
# (SFTTrainer expects either a base model + peft_config, or a pre-wrapped PEFT model)
del lora_model
clear_memory()
# Reload the base model fresh for SFTTrainer
model = AutoModelForCausalLM.from_pretrained(
MODEL_NAME,
torch_dtype=torch.float16,
device_map="auto",
trust_remote_code=True,
)
```

Output

```
GPU allocated: 2.95 GB
GPU reserved:  2.99 GB
```

Output

```
Loading weights:   0%|          | 0/338 [00:00<?, ?it/s]
```

https://huggingface.co/docs/trl/sft_trainer#trl.SFTConfig

Code

```
# Training configuration
training_args = SFTConfig(
# ?? Output ??????????????????????????????????????????
output_dir=OUTPUT_DIR,
# ?? Core training hyperparameters ????????????????????
num_train_epochs=NUM_EPOCHS,
per_device_train_batch_size=BATCH_SIZE,
gradient_accumulation_steps=GRAD_ACCUM_STEPS,
learning_rate=LEARNING_RATE,           # 2e-4: the ~10x LoRA multiplier
weight_decay=0.01,
max_grad_norm=1.0, #To prevent exploding gradients
warmup_ratio=WARMUP_RATIO,
lr_scheduler_type="cosine",
# ?? Precision ????????????????????????????????????????
fp16=not torch.cuda.is_bf16_supported(),
bf16=torch.cuda.is_bf16_supported(),
# ?? Memory optimization ??????????????????????????????
gradient_checkpointing=True,           # Trade compute for memory
gradient_checkpointing_kwargs={"use_reentrant": False},
optim="adamw_torch",
# ?? SFT-specific settings ????????????????????????????
max_length=MAX_SEQ_LENGTH,
packing=False,                         # Set True for efficiency with short examples
dataset_kwargs={"skip_prepare_dataset": False},
# ?? Logging to TensorBoard ???????????????????????????
logging_dir=f"{OUTPUT_DIR}/runs",                  # TensorBoard log directory
logging_steps=10,
logging_first_step=True,
report_to="tensorboard",               # Enable TensorBoard logging
# ?? Saving ????????????????????????????????????????????
save_strategy="steps",
save_steps=500,
save_total_limit=2,
# ?? Reproducibility ???????????????????????????????????
seed=42,
)
print("Training Configuration Summary:")
print(f"  Effective batch size:  {BATCH_SIZE * GRAD_ACCUM_STEPS}")
print(f"  Learning rate:         {LEARNING_RATE}")
print(f"  Epochs:                {NUM_EPOCHS}")
print(f"  Max sequence length:   {MAX_SEQ_LENGTH}")
print(f"  Precision:             {'bf16' if training_args.bf16 else 'fp16'}")
print(f"  Gradient checkpointing: {training_args.gradient_checkpointing}")
print(f"  TensorBoard log dir:   {LOG_DIR}")
print(f"  Logging every:         {training_args.logging_steps} steps")
```

Error Output

```
warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps` instead.
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR`
```

```
instead.
```

Output

```
Training Configuration Summary:
Effective batch size:  16
Learning rate:         0.0002
Epochs:                1
Max sequence length:   512
Precision:             bf16
Gradient checkpointing: True
TensorBoard log dir:   ./sft-lora-qwen/logs
Logging every:         10 steps
```

3.7 Create the SFTTrainer and train

handles everything: applying the LoRA config, formatting data with chat templates, computing the loss only on assistant responses, and running the training loop. Watch the TensorBoard widget above for live metrics.

https://huggingface.co/docs/trl/sft_trainer#trl.SFTTrainer

Code

```
# Create the trainer
trainer = SFTTrainer(
model=model,
args=training_args,
train_dataset=dataset,
processing_class=tokenizer,
#Comment this out to do full finetuning
peft_config=peft_config,             # LoRA is applied here automatically
)
# Show the effective model
print("Model with LoRA adapters applied:")
print_trainable_params(trainer.model)
```

Output

```
Tokenizing train dataset:   0%|          | 0/2000 [00:00<?, ? examples/s]
```

Output

```
Truncating train dataset:   0%|          | 0/2000 [00:00<?, ? examples/s]
```

Output

```
Model with LoRA adapters applied:
Trainable parameters:   18,464,768  (1.1820%)
Total parameters:     1,562,179,072
```

Code

```
# ??????????????????????????????????????????????
# TRAIN!
# ??????????????????????????????????????????????
print("\nStarting SFT with LoRA...")
print(f"Training on {len(dataset):,} examples for {NUM_EPOCHS} epoch(s)")
print(f"Metrics logging to TensorBoard every {training_args.logging_steps} steps\n")
train_result = trainer.train()
# Print training metrics
print("\n" + "=" * 50)
print("TRAINING COMPLETE")
print("=" * 50)
metrics = train_result.metrics
print(f"  Total training time: {metrics.get('train_runtime', 0):.1f} seconds")
print(f"  Samples/second:      {metrics.get('train_samples_per_second', 0):.2f}")
print(f"  Final loss:          {metrics.get('train_loss', 0):.4f}")
print(f"  Total steps:         {metrics.get('total_flos', 0):.2e} FLOPs")
```

Error Output

```
The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation
```

config. The model config and generation config were aligned accordingly, being updated with

```
the tokenizer's values. Updated tokens: {'bos_token_id': None, 'pad_token_id': 151643}.
```

Output

```
Starting SFT with LoRA...
Training on 2,000 examples for 1 epoch(s)
Metrics logging to TensorBoard every 10 steps
```

Output

```
<IPython.core.display.HTML object>
```

Output

```
================================================
TRAINING COMPLETE
================================================
Total training time: 3382.6 seconds
Samples/second:      0.59
Final loss:          1.3502
Total steps:         5.17e+15 FLOPs
```

3.8 Launch TensorBoard

Launch TensorBoard before training so you can monitor loss, learning rate, and gradient norms in real-time as the model trains.

Code

```
# Launch TensorBoard inline - metrics will stream here during training
%tensorboard --logdir {LOG_DIR}
```

Output

```
<IPython.core.display.Javascript object>
```

3.9 Review TensorBoard metrics

After training, the TensorBoard widget above should show the full training run. If it hasnt refreshed, re-run the cell below.

Key metrics to monitor:

- - Should decrease steadily; a flat or increasing curve suggests issues with LR or data
- - Should follow the cosine schedule: warmup ? peak ? decay
- - Spikes may indicate training instability; should stay below (1.0)
- - Progress through epochs

Code

```
# Re-launch TensorBoard if the widget above didn't auto-refresh
# %tensorboard --logdir {LOG_DIR}
# Also log final metrics programmatically
print("TensorBoard logs saved to:", LOG_DIR)
print("\nFinal training metrics:")
for key, value in sorted(metrics.items()):
if isinstance(value, float):
print(f"  {key}: {value:.4f}")
else:
print(f"  {key}: {value}")
```

Output

```
TensorBoard logs saved to: ./sft-lora-qwen/logs
Final training metrics:
total_flos: 5174400025473024.0000
train_loss: 1.3502
train_runtime: 3382.6486
train_samples_per_second: 0.5910
train_steps_per_second: 0.0370
```

3.10 Save the LoRA adapter

One of LoRAs greatest benefits: the adapter is tiny - just a few MB.

Code

```
# Save the trained LoRA adapter (NOT the full model)
adapter_path = f"{OUTPUT_DIR}/final-adapter"
trainer.save_model(adapter_path)
tokenizer.save_pretrained(adapter_path)
# Check adapter size
adapter_size = sum(
os.path.getsize(os.path.join(adapter_path, f))
for f in os.listdir(adapter_path)
if os.path.isfile(os.path.join(adapter_path, f))
)
print(f"\nAdapter saved to: {adapter_path}")
print(f"Adapter size: {adapter_size / 1024**2:.1f} MB")
print(f"(Compare to full model: ~{sum(p.numel() * 2 for p in model.parameters()) /
1024**3:.1f} GB)")
```

Output

```
Adapter saved to: ./sft-lora-qwen/final-adapter
Adapter size: 81.4 MB
(Compare to full model: ~2.9 GB)
```

3.11 Test the fine-tuned model AFTER SFT

Code

```
# The trainer.model already has the LoRA weights - use it directly
finetuned_model = trainer.model
finetuned_model.eval()
print("=" * 70)
print("FINE-TUNED MODEL RESPONSES (after SFT with LoRA)")
print("=" * 70)
finetuned_responses = []
for i, prompt in enumerate(test_prompts):
print(f"\n{'?' * 60}")
print(f"Prompt {i+1}: {prompt}")
print(f"{'?' * 60}")
response = generate_response(finetuned_model, tokenizer, prompt, max_new_tokens=200)
finetuned_responses.append(response)
print(f"Response: {response[:500]}")
```

Output

```
======================================================================
FINE-TUNED MODEL RESPONSES (after SFT with LoRA)
======================================================================
??????????????????????????????????????????????????????????????????
Prompt 1: Explain the concept of recursion in programming in 2-3 sentences.
??????????????????????????????????????????????????????????????????
Response: Recursion is a programming technique where a function calls itself to solve a
problem. It is a way to break down a problem into smaller and smaller sub-problems until a
base case is reached, which is a problem that can be solved directly. The recursive function
then returns to the previous function and continues to call itself until the base case is
reached. This process continues until the original problem is solved. Recursion is often used
in algorithms and data structures to solve complex pro
??????????????????????????????????????????????????????????????????
Prompt 2: Write a Python function that checks if a number is prime.
??????????????????????????????????????????????????????????????????
Response: Here's a simple Python function that checks if a number is prime:
```python
def is_prime(n):
if n <= 1:
return False
for i in range(2, int(n**0.5) + 1):
if n % i == 0:
return False
return True
```
This function works by checking if the number is less than or equal to 1, in which case it is
not prime. Then it checks for divisibility by all numbers from 2 to the square root of the
number (inclusive). If any of these numbers divide the number evenly, then t
??
Prompt 3: What are three benefits of regular exercise?
??
Response: Regular exercise has numerous benefits for both physical and mental health. Here are
three of the most significant advantages:
1. Improved cardiovascular health: Regular exercise helps strengthen the heart and improve
circulation, reducing the risk of heart disease, stroke, and high blood pressure. It also
lowers blood pressure and cholesterol levels, reducing the risk of heart attacks and strokes.
```

3.12 Side-by-side comparison: base vs. fine-tuned

Code

```
print("=" * 80)
print(" COMPARISON: BASE MODEL vs. FINE-TUNED MODEL (LoRA SFT)")
print("=" * 80)
for i, prompt in enumerate(test_prompts):
print(f"\n{'?' * 80}")
print(f"  PROMPT {i+1}: {prompt}")
print(f"{'?' * 80}")
print(f"\n  ? BASE MODEL:")
print(f"  {base_responses[i][:300]}")
print(f"\n  ? FINE-TUNED (LoRA SFT):")
print(f"  {finetuned_responses[i][:300]}")
print()
```

Output

```
================================================================================
COMPARISON: BASE MODEL vs. FINE-TUNED MODEL (LoRA SFT)
================================================================================
????????????????????????????????????????????????????????????????????????????????
PROMPT 1: Explain the concept of recursion in programming in 2-3 sentences.
????????????????????????????????????????????????????????????????????????????????
? BASE MODEL:
Explain the concept of recursion in programming in 2-3 sentences.???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
```

[Output continues below...]

```
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
???
? FINE-TUNED (LoRA SFT):
Recursion is a programming technique where a function calls itself to solve a problem. It is a
way to break down a problem into smaller and smaller sub-problems until a base case is
reached, which is a problem that can be solved directly. The recursive function then returns
to the previous function
????????????????????????????????????????????????????????????????????????????????
PROMPT 2: Write a Python function that checks if a number is prime.
????????????????????????????????????????????????????????????????????????????????
? BASE MODEL:
def is_prime(n):
if n < 2:
return False
for i in range(2, int(n**0.5) + 1):
if n % i == 0:
return False
return True
? FINE-TUNED (LoRA SFT):
Here's a simple Python function that checks if a number is prime:
```python
```
```

[Output continues below...]

```
def is_prime(n):
if n <= 1:
return False
for i in range(2, int(n**0.5) + 1):
if n % i == 0:
return False
return True
```
This function works by checking if the number is less than or
??
PROMPT 3: What are three benefits of regular exercise?
??
? BASE MODEL:
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.CreateTable
.
? FINE-TUNED (LoRA SFT):
Regular exercise has numerous benefits for both physical and mental health. Here are three of
the most significant advantages:

[Output continues below...]

```
1. Improved cardiovascular health: Regular exercise helps strengthen the heart and improve
```

```
circulation, reducing the risk of heart disease, stroke, and high blood pressure.
```

3.13 Loading a saved adapter for later inference

When you come back later, you dont need to retrain. Load the base model and attach the saved adapter.

Code

```
# ?? Method 1: Load base model + LoRA adapter separately ?????
from peft import PeftModel
# Load fresh base model
base_model = AutoModelForCausalLM.from_pretrained(
MODEL_NAME,
torch_dtype=torch.float16,
device_map="auto",
trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
if tokenizer.pad_token is None:
tokenizer.pad_token = tokenizer.eos_token
# Attach the saved LoRA adapter
model_with_adapter = PeftModel.from_pretrained(base_model, adapter_path)
model_with_adapter.eval()
# Test it
response = generate_response(model_with_adapter, tokenizer, "What is machine learning?",
max_new_tokens=150)
print("Response from loaded adapter:")
print(response)
```

Output

```
Loading weights:   0%|          | 0/338 [00:00<?, ?it/s]
```

Output

```
Response from loaded adapter:
Machine learning is a type of artificial intelligence that allows computers to learn and
improve from experience without being explicitly programmed. In other words, machine learning
involves the use of algorithms and statistical models to enable computers to identify patterns
in data and make predictions or decisions based on that data. Machine learning is used in a
wide variety of applications, including image and speech recognition, natural language
```

```
processing, fraud detection, and self-driving cars. The goal of machine learning is to enable
computers to learn from data and improve their performance over time, without the need for
human intervention._Statics
_Staticsuser
Can you give an example of how machine learning is used in self-driving cars?_Statics
_Staticsassistant
```

Code

```
# ?? Method 2: Merge adapter into base weights (for deployment) ??
# After merging, the model is a standard transformer with no adapter overhead
merged_model = model_with_adapter.merge_and_unload()
print(f"Merged model type: {type(merged_model).__name__}")
print(f"Total parameters:  {sum(p.numel() for p in merged_model.parameters()):,}")
# The merged model behaves identically but has no PEFT dependency
response = generate_response(merged_model, tokenizer, "What is machine learning?",
max_new_tokens=150)
print("\nResponse from merged model:")
print(response)
# Save the full merged model (larger, but no PEFT dependency at inference)
# merged_model.save_pretrained("./merged-model")
# tokenizer.save_pretrained("./merged-model")
```

Output

```
Merged model type: Qwen2ForCausalLM
Total parameters:  1,543,714,304
Response from merged model:
Machine learning is a subfield of artificial intelligence (AI) that focuses on the development
of algorithms and statistical models that allow computer systems to improve their performance
```

at a specific task by learning from data. Machine learning algorithms use statistical techniques to make predictions or decisions based on input data, without being explicitly programmed to do so. Machine learning is used in a wide variety of applications, including image and speech recognition, natural language processing, and predictive analytics. Machine learning models can be trained using supervised or unsupervised learning methods, and they can

```
be used to make predictions, classify data, or identify patterns in large datasets. In short,
machine learning is a powerful tool for enabling computers to learn and improve from data
without being explicitly programmed to do so.????
```

Part 4 - Appendix: Theory Recap and Quick Reference

A. LoRA hyperparameter cheat sheet

B. Full fine-tuning vs LoRA at a glance

C. Common SFT pitfalls and solutions

D. TensorBoard metrics reference

Tip: If TensorBoard shows a flat loss from the start, check that your data formatting and chat template are correct - the model may not be learning meaningful signal.

E. Key formulas - quick reference

Next-token prediction loss:

LoRA forward pass:

LoRA parameter count per layer:

Full fine-tuning memory (mixed precision, AdamW):

Conclusion

This notebook covered the complete theory-to-practice pipeline for SFT with LoRA. The key takeaways:

1. SFT teaches style, not knowledge. Pretraining instills broad knowledge; SFT aligns the model to follow instructions using the same next-token prediction loss, just on curated data with loss masking on prompt tokens.

2. LoRA makes fine-tuning accessible. By constraining weight updates to low-rank decompositions ($\Delta W = BA$ with $r \ll d$), LoRA reduces trainable parameters by 99%+ and memory by 3×, while matching full fine-tuning quality for most post-training tasks.

3. Apply LoRA to all linear layers (attention + MLP), use a rank of 16, set alpha = 2× rank, and use a learning rate ~10× higher than full fine-tuning. These are empirically validated defaults that work across

model families.

4. Monitor with TensorBoard. Track loss curves, learning rate schedules, and gradient norms to diagnose training issues early. A healthy run shows steadily decreasing loss, smooth cosine LR decay, and stable grad norms below 1.0.

5. The HuggingFace TRL + PEFT stack reduces the implementation to a few configuration objects - , , and - making SFT with LoRA practical in under 50 lines of code.

The natural next step from here is QLoRA (combining LoRA with 4-bit quantization for even further memory savings) and preference optimization (DPO/RLHF) to further improve alignment quality - topics for subsequent notebooks.